Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería en Software | Sistemas Distribuidos

FORM-FB-01 — Fichas bibliográficas

Estudiante: Calderón Saltos Joseph Alexander

Ficha 1. Pruebas de fallos en sistemas distribuidos

Referencia en formato IEEE (BibLaTeX)

[1] E. B. Gulcan et al., «Model-Guided Fuzzing of Distributed Systems,» *Proceedings of the ACM on Programming Languages*, vol. 9, n.º OOPSLA2, 2025, **artno** 282. DOI: 10.1145/3763060

DOI verificado

DOI: 10.1145/3763060

Resumen

En los sistemas distribuidos, algunos errores no aparecen siempre: dependen del orden en que ocurren ciertos eventos. Para encontrarlos, esta investigación utiliza ModelFuzz, una técnica que guía las pruebas con modelos formales escritos en TLA+. En lugar de limitarse a las pruebas aleatorias, busca explorar distintas situaciones del sistema. Para evaluar el método, se hicieron pruebas con Etcd-raft y RedisRaft, dos implementaciones del algoritmo Raft. Los resultados muestran que ModelFuzz permitió cubrir más situaciones y detectar fallos en menos tiempo que las estrategias comparadas. Así, un servicio puede parecer que funciona bien y aun así presentar errores bajo ciertas condiciones.

Dato concreto de la fuente y ubicación

Los autores informan **12 errores antes desconocidos**, de los cuales **4** solo se detectaron mediante las pruebas guiadas por los modelos. *Ubicación: resumen e introducción, [1, pp. 1–2] (versión final de ACM).*

Nota personal: aporte al futuro informe

Lo que más me interesa de esta lectura es que un error puede pasar desapercibido hasta que ocurre una combinación particular de eventos. Además, si más adelante pruebo etcd, quiero revisar qué estado mantiene el clúster después de un fallo y no limitarme a mostrar que los contenedores siguen encendidos. La investigación trabaja con Etcd-raft, que no equivale a estudiar todas las funciones de etcd.

Ficha 2. Elección de líder y rendimiento de Raft

Referencia en formato IEEE (BibLaTeX)

[2] Z. Du et al., «Multi-strategy-based leader election mechanism for the Raft algorithm,» *Concurrency and Computation: Practice and Experience*, vol. 35, n.º 22, 2023, **artno** e7734. DOI: 10.1002/cpe.7734

DOI verificado

DOI: 10.1002/cpe.7734

Resumen

Un servidor puede seguir siendo el líder de un clúster Raft aunque responda con lentitud. Ese es el problema que aborda esta investigación. La propuesta modifica el mecanismo de elección para tener en cuenta el rendimiento de los servidores y dar prioridad a determinados candidatos. También contempla iniciar otra elección cuando las condiciones lo requieren. Para evaluarla, los investigadores analizaron sus propiedades e hicieron experimentos con diferentes cantidades de servidores. Al compararla con el Raft original, observaron menos repeticiones de elección por división del voto y un menor tiempo promedio de elección. Aun así, el nuevo mecanismo también requiere cálculos adicionales.

Dato concreto de la fuente y ubicación

En el experimento con **15 servidores** y **200 elecciones**, los investigadores registraron **19 repeticiones de elección por división del voto** con Raft original; también informan una reducción aproximada del **10 %** en el tiempo promedio de elección con su propuesta. *Ubicación: p. 2, [2, p. 2], y sección 5.3, [2, pp. 13–14] (PDF de Wiley).*

Nota personal: aporte al futuro informe

Me quedó más clara la diferencia entre tener un líder disponible y tener uno con buen rendimiento. Eso será útil cuando observe una elección de líder en Docker. También voy a tener que distinguir el funcionamiento normal de Raft de los cambios propuestos por los investigadores. No sería correcto decir que etcd o Consul incluyen esas mejoras sin comprobarlo.

Ficha 3. Verificación formal de protocolos de consenso

Referencia en formato IEEE (BibLaTeX)

[3] Z. Feng y R. Zhang, «Survey of Consensus Protocol's Formal Verification,» *Cybersecurity*, vol. 9, 2026, **artno** 81. DOI: 10.1186/s42400-025-00501-2

DOI verificado

DOI: 10.1186/s42400-025-00501-2

Resumen

La verificación formal permite comprobar propiedades de los protocolos de consenso sin depender únicamente de pruebas prácticas. Esta revisión reúne investigaciones que utilizan métodos como la comprobación de modelos y la demostración de teoremas. También explica qué propiedades buscan verificar y qué dificultades encuentran al aplicar esas técnicas. Entre los protocolos estudiados aparece Raft, que está diseñado para tolerar fallos no bizantinos. El trabajo no se centra en probar un clúster específico de etcd o Consul, sino en mostrar cómo se analizan las garantías de un protocolo. Así se entiende mejor la diferencia entre observar un resultado y comprobar formalmente ciertas propiedades.

Dato concreto de la fuente y ubicación

La **Tabla 1** incluye a **Raft** entre los protocolos para los que se ha utilizado *model checking* y enumera propiedades examinadas como **seguridad, consistencia y progreso** (*safety, consistency, liveness*). *Ubicación: Tabla 1, [3, p. 5], PDF de Springer Nature.*

Nota personal:

La fuente sirvió para no confundir una prueba práctica con una demostración formal. Si detengo un nodo y el sistema sigue funcionando, habré observado un comportamiento concreto, pero no habré demostrado todas las propiedades de seguridad. Esa diferencia puede ayudarme a explicar mejor los límites del experimento del próximo informe.

Ficha 4. Medición de rendimiento en etcd y Hyperledger Fabric

Referencia en formato IEEE (BibLaTeX)

[4] L. Xiao et al., «Scheme for improving RAFT-based blockchain performance,» en *Proceedings of the 2023 International Conference on Software Engineering and Machine Learning*, ép. Applied and Computational Engineering, vol. 27, EWA Publishing, 2023, págs. 81-89. DOI: 10.54254/2755-2721/27/20230173

DOI verificado

DOI: 10.54254/2755-2721/27/20230173

Resumen

Este trabajo compara el rendimiento de etcd y Hyperledger Fabric, dos plataformas evaluadas en relación con mecanismos de consenso basados en Raft. Para hacerlo, los investigadores midieron la cantidad de operaciones completadas, el tiempo de respuesta y la robustez durante las pruebas. También variaron factores como el número de clientes, claves y conexiones, y observaron diferencias entre lecturas y escrituras. En los escenarios examinados, etcd obtuvo mejores resultados de rendimiento que Hyperledger Fabric. Además, las lecturas rindieron mejor que las escrituras en ambas plataformas. Estos resultados corresponden a las configuraciones probadas y no significan que suceda lo mismo en cualquier clúster.

Dato concreto de la fuente y ubicación

El estudio compara **dos plataformas** —etcd y Hyperledger Fabric— y concluye, para sus pruebas, que **las lecturas rindieron mejor que las escrituras** en ambas. *Ubicación: resumen, [4, p. 81]; publicación completa, pp. 81–89.*

Nota personal:

Aquí se trabaja con etcd, así que el ejemplo me resulta cercano a la opción que se está considerando. Me parece útil ver que los resultados dependen de qué se mide y de cómo se configura la prueba. Cuando haga mi experimento, tendré que separar dos cosas: comprobar la continuidad del servicio y medir el rendimiento de sus operaciones.

Ficha 5. Cómo evaluar sistemas de coordinación distribuida

Referencia en formato IEEE (BibLaTeX)

[5] B. Turkkan et al., «How to Evaluate Distributed Coordination Systems?—A Survey and Analysis,» *IEEE Transactions on Parallel and Distributed Systems*, vol. 37, n.º 1, págs. 198-212, 2026. DOI: 10.1109/TPDS.2025.3631614

DOI verificado

DOI: 10.1109/TPDS.2025.3631614

Resumen

Medir solo cuántas operaciones procesa un sistema no basta para compararlo con otro. Esta revisión plantea que también hay que examinar la consistencia, la disponibilidad y la respuesta ante fallos. Para mostrarlo, organizaron métodos, parámetros y herramientas utilizadas en la evaluación de sistemas de coordinación distribuida. Entre las herramientas comparadas aparecen YCSB y Jepsen, que no evalúan exactamente los mismos aspectos. Los autores señalan limitaciones de los bancos de pruebas disponibles y explican por qué una comparación puede ser incompleta si utiliza pocas métricas. Por eso este enfoque sirve para diseñar experimentos con objetivos claros y para interpretar correctamente lo observado.

Dato concreto de la fuente y ubicación

La **Tabla 5** distingue métricas de rendimiento, escalabilidad, disponibilidad y consistencia. Allí **YCSB** aparece asociado a rendimiento (*throughput* y latencia), mientras que **Jepsen** incluye pruebas de fallos de nodos, particiones y comprobaciones de consistencia. *Ubicación: Tabla 5, p. 11 de la versión gratuita en arXiv correspondiente a [5]; esta página no se atribuye a la edición final IEEE.*

Nota personal:

Básicamente no basta con comprobar si un sistema distribuido funciona, también hay que observar cómo responde cuando ocurre algún problema. Esto me servirá para el próximo informe, porque podré analizar el comportamiento de etcd o Consul que estoy considerando y explicar sus resultados tomando en cuenta aspectos como la disponibilidad, la consistencia y la tolerancia a fallos.

Declaración de uso de inteligencia artificial. Se utilizó ChatGPT como apoyo para organizar las fuentes, preparar los borradores de las fichas y escribir el código LaTeX. Para verificar la información, comprobé los DOI en las páginas de las publicaciones y contrasté los datos y las secciones citadas con los artículos originales.